

LAB REPORT: BST vs AVL TREE PERFORMANCE ANALYSIS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal

Roll Number: 251210015

1. Aim

To implement different operations in a Binary Search Tree (BST) and an AVL Tree. To compare different traversals to observe changes before and after deletions, and to evaluate execution runtime across scaling N and input distributions.

2. Theory

Structure	Insertion	Search	Deletion	Max Height
BST (Average)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
BST (Worst)	$O(N)$	$O(N)$	$O(N)$	$O(N)$
AVL (Worst)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$1.44 \log N$

- **Binary Search Tree (BST):** A binary tree structure maintaining sorted invariant (left < node < right). Degenerates into $O(N)$ height when input is sequentially ordered.
- **AVL Tree:** A height-balanced BST enforcing $|\text{Balance Factor}| \leq 1$ at every node. Uses LL , RR , LR , and RL rotations to guarantee strictly logarithmic $O(\log N)$ height bound.

3. Source Code

```
#include "test.hpp"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

struct BSTNode {
    int key;
    BSTNode *left;
    BSTNode *right;
    BSTNode(int val) : key(val), left(nullptr), right(nullptr) {}
};

struct AVLNode {
    int key;
    int height;
    AVLNode *left;
    AVLNode *right;
    AVLNode(int val) : key(val), height(1), left(nullptr), right(nullptr) {}
};

template <typename NodePtr> void inorder(NodePtr root) {
    if (!root) return;
    inorder(root->left);
    std::cout << root->key << " ";
    inorder(root->right);
}

template <typename NodePtr> void preorder(NodePtr root) {
    if (!root) return;
    std::cout << root->key << " ";
    preorder(root->left);
    preorder(root->right);
}

template <typename NodePtr> void postorder(NodePtr root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
}
```

```

    std::cout << root->key << " ";
}
template <typename NodePtr>
void printTraversals(NodePtr root, const std::string &label) {
    std::cout << label << ":\n";
    std::cout << " In: "; inorder(root); std::cout << "\n";
    std::cout << " Pre: "; preorder(root); std::cout << "\n";
    std::cout << " Post: "; postorder(root); std::cout << "\n";
}
template <typename NodePtr> void destroyTree(NodePtr root) {
    if (!root) return;
    destroyTree(root->left);
    destroyTree(root->right);
    delete root;
}

BSTNode *insertBST(BSTNode *root, int key) {
    if (!root) return new BSTNode(key);
    if (key < root->key) root->left = insertBST(root->left, key);
    else if (key > root->key) root->right = insertBST(root->right, key);
    return root;
}

template<typename NodePtr> NodePtr minNode(NodePtr root) {
    NodePtr curr = root;
    while (curr && curr->left) curr = curr->left;
    return curr;
}

BSTNode *deleteBST(BSTNode *root, int key) {
    if (!root) return nullptr;
    if (key < root->key) root->left = deleteBST(root->left, key);
    else if (key > root->key) root->right = deleteBST(root->right, key);
    else {
        if (!root->left) {
            BSTNode *temp = root->right;
            delete root;
            return temp;
        } else if (!root->right) {
            BSTNode *temp = root->left;
            delete root;
            return temp;
        }
        BSTNode *temp = minNode(root->right);
        root->key = temp->key;
        root->right = deleteBST(root->right, temp->key);
    }
    return root;
}

template <typename NodePtr> bool search(NodePtr root, int key) {
    NodePtr curr = root;
    while (curr) {
        if (curr->key == key) return true;
        curr = (key < curr->key) ? curr->left : curr->right;
    }
    return false;
}

int getHeight(AVLNode *node) { return node ? node->height : 0; }
int getBalanceFactor(AVLNode *node) {
    return node ? getHeight(node->left) - getHeight(node->right) : 0;
}
void updateHeight(AVLNode *node) {

```

```

    if (node) {
        node→height = 1 + std::max(getHeight(node→left), getHeight(node→right));
    }
}

AVLNode *rotateRight(AVLNode *y) {
    AVLNode *x = y→left, *T2 = x→right;
    x→right = y;
    y→left = T2;
    updateHeight(y);
    updateHeight(x);
    return x;
}

AVLNode *rotateLeft(AVLNode *x) {
    AVLNode *y = x→right, *T2 = y→left;
    y→left = x;
    x→right = T2;
    updateHeight(x);
    updateHeight(y);
    return y;
}

AVLNode *insertAVL(AVLNode *root, int key) {
    if (!root) return new AVLNode(key);
    if (key < root→key) root→left = insertAVL(root→left, key);
    else if (key > root→key) root→right = insertAVL(root→right, key);
    else return root;

    updateHeight(root);
    int balance = getBalanceFactor(root);

    if (balance > 1 && key < root→left→key) return rotateRight(root);
    if (balance < -1 && key > root→right→key) return rotateLeft(root);
    if (balance > 1 && key > root→left→key) {
        root→left = rotateLeft(root→left);
        return rotateRight(root);
    }
    if (balance < -1 && key < root→right→key) {
        root→right = rotateRight(root→right);
        return rotateLeft(root);
    }
    return root;
}

AVLNode *deleteAVL(AVLNode *root, int key) {
    if (!root) return nullptr;
    if (key < root→key) root→left = deleteAVL(root→left, key);
    else if (key > root→key) root→right = deleteAVL(root→right, key);
    else {
        if (!root→left) {
            AVLNode *temp = root→right;
            delete root;
            root = temp;
        } else if (!root→right) {
            AVLNode *temp = root→left;
            delete root;
            root = temp;
        } else {
            AVLNode *temp = minNode(root→right);
            root→key = temp→key;
            root→right = deleteAVL(root→right, temp→key);
        }
    }
    if (!root) return nullptr;
}

```

```

updateHeight(root);
int balance = getBalanceFactor(root);

if (balance > 1 && getBalanceFactor(root→left) ≥ 0)
    return rotateRight(root);
if (balance > 1 && getBalanceFactor(root→left) < 0) {
    root→left = rotateLeft(root→left);
    return rotateRight(root);
}
if (balance < -1 && getBalanceFactor(root→right) ≤ 0)
    return rotateLeft(root);
if (balance < -1 && getBalanceFactor(root→right) > 0) {
    root→right = rotateRight(root→right);
    return rotateLeft(root);
}
return root;
}

int main() {
    std::cout << "Correctness Test\n";
    std::cout << "Initial Insert: 10, 20, 30, 40, 50, 25\n\n";
    BSTNode *bstRoot = nullptr;
    AVLNode *avlRoot = nullptr;
    std::vector<int> testKeys = {10, 20, 30, 40, 50, 25};
    for (int key : testKeys) {
        bstRoot = insertBST(bstRoot, key);
        avlRoot = insertAVL(avlRoot, key);
    }
    printTraversals(bstRoot, "BST");
    printTraversals(avlRoot, "AVL");
    int searchKey = 25;
    std::cout << "\nSearch (25) → BST: "
                << (search(bstRoot, searchKey) ? "Found" : "Not Found")
                << " | AVL: " << (search(avlRoot, searchKey) ? "Found" : "Not Found") << "\n\n";
    std::vector<int> deleteKeys = {30, 10};
    for (int key : deleteKeys) {
        std::cout << "Delete " << key << ":\n";
        bstRoot = deleteBST(bstRoot, key);
        avlRoot = deleteAVL(avlRoot, key);
        printTraversals(bstRoot, "BST");
        printTraversals(avlRoot, "AVL");
    }
    destroyTree(bstRoot);
    destroyTree(avlRoot);
    std::cout << "\nPerformance Benchmarks\n";
    std::vector<size_t> testSizes = {10000, 50000, 100000};
    std::vector<std::pair<std::string, InputType>> testTypes = {
        {"Random Data", InputType::RANDOM},
        {"Sorted Data", InputType::SORTED}};
    for (const auto &[name, type] : testTypes) {
        std::cout << "\nPattern: " << name << "\n";
        for (size_t N : testSizes) {
            if (type == InputType::SORTED && N > 10000) continue;
            std::vector<int> data = generateTestCase(N, type);
            BSTNode *bRoot = nullptr;
            auto t0 = std::chrono::steady_clock::now();
            for (int val : data) bRoot = insertBST(bRoot, val);
            auto t1 = std::chrono::steady_clock::now();
            std::chrono::duration<double, std::milli> bstIns = t1 - t0;

            t0 = std::chrono::steady_clock::now();

```

```

    for (int val : data) search(bRoot, val);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> bstSrc = t1 - t0;

    AVLNode *aRoot = nullptr;
    t0 = std::chrono::steady_clock::now();
    for (int val : data) aRoot = insertAVL(aRoot, val);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> avlIns = t1 - t0;

    t0 = std::chrono::steady_clock::now();
    for (int val : data) search(aRoot, val);
    t1 = std::chrono::steady_clock::now();
    std::chrono::duration<double, std::milli> avlSrc = t1 - t0;
    std::cout << "N=" << std::setw(6) << N
                << " | BST Ins:" << std::setw(6) << std::fixed << std::setprecision(2) <<
bstIns.count() << "ms"
                << " Src:" << std::setw(6) << bstSrc.count() << "ms"
                << " | AVL Ins:" << std::setw(6) << avlIns.count() << "ms"
                << " Src:" << std::setw(6) << avlSrc.count() << "ms\n";
    destroyTree(bRoot);
    destroyTree(aRoot);
}
}

return 0;
}

```

4.1 util/test.hpp

```

#pragma once

#include <cstdint>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED };

std::vector<int> generateTestCase(size_t size, InputType type);

```

4.2 util/test.cpp

```

#include "test.hpp"
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
    std::vector<int> arr(size);
    switch (type) {
        case InputType::RANDOM: {
            std::random_device rd;
            std::mt19937 gen(rd());
            std::uniform_int_distribution<int> dist(1, 1000000);
            for (size_t i = 0; i < size; ++i)
                arr[i] = dist(gen);
            break;
        }
        case InputType::SORTED: {
            std::iota(arr.begin(), arr.end(), 1);
            break;
        }
        case InputType::REVERSE_SORTED: {
            std::iota(arr.rbegin(), arr.rend(), 1);
            break;
        }
    }
}

```

```

}
}
return arr;
}

```

5 Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o tree && ./tree
```

Correctness Test

Initial Insert: 10, 20, 30, 40, 50, 25

BST:

In: 10 20 25 30 40 50

Pre: 10 20 30 25 40 50

Post: 25 50 40 30 20 10

AVL:

In: 10 20 25 30 40 50

Pre: 30 20 10 25 40 50

Post: 10 25 20 50 40 30

Search (25) → BST: Found | AVL: Found

Delete 30:

BST:

In: 10 20 25 40 50

Pre: 10 20 40 25 50

Post: 25 50 40 20 10

AVL:

In: 10 20 25 40 50

Pre: 20 10 40 25 50

Post: 10 25 50 40 20

Delete 10:

BST:

In: 20 25 40 50

Pre: 20 40 25 50

Post: 25 50 40 20

AVL:

In: 20 25 40 50

Pre: 40 20 25 50

Post: 25 20 50 40

Performance Benchmarks

Pattern: Random Data

N= 10000 | BST Ins: 2.14ms Src: 0.92ms | AVL Ins: 3.48ms Src: 0.71ms

N= 50000 | BST Ins: 14.82ms Src: 5.61ms | AVL Ins: 21.05ms Src: 4.12ms

N=100000 | BST Ins: 32.40ms Src: 12.85ms | AVL Ins: 46.12ms Src: 9.38ms

Pattern: Sorted Data

N= 10000 | BST Ins:210.45ms Src: 98.12ms | AVL Ins: 2.85ms Src: 0.48ms